Introduction to Programming

Lecture 16

VU
Virtual University

In Today Lecture

- Conclude the last discussion
- Multi-dimensional Arrays
- Pointers to Pointers

Example 1

```
char myName [ ] = “Amir” ;
char *myNamePtr = “Amir” ;
```

Multi-dimensional Arrays

```
char multi [ 5 ] [ 10 ] ;
```

Multi-dimensional Array in Memory

Placed sequentially in the memory

[0]	[1]	[2]	[3]	[4]	[0]	[1]	[2]	[3]	[4]	[0]	[1]	[2]	[3]	[4]
1	2	3	4	10	7	9	11	14	10	17	25	39	45	58

1st row
1st col

2nd row
1st col

3rd row
1st col

Dereferencing array element

multi [2] [3]

*multi

?

Example 2

```
#include<iostream.h>
main ( )
{
 char multi [ 5 ] [ 10 ] ;
 cout << multi << endl ;
 cout << *multi ;
 cout << **multi ;
}
```

multi + 3

*(multi + 3)

*(multi + 3) + 3

((multi + 3) + 3)

Example 3

```
main ( )
{
    int multi [ 5 ] [ 6 ] ;
    int row , col ;
    int *ptr ;
    ptr = *multi ;
    for ( row = 0 ; row < 5 ; row ++ )
    {
        for ( col = 0 ; col < 10 ; col ++ )
        {
            multi [ row ] [ col ] = row * col ;
        }
    }
```

Example 3

```
for ( row = 0 ; row < 5 ; row ++ )
 {
    for ( col = 0 ; col < 10 ; col ++)
    {

      cout << *( ptr ++ ) << " ";
    }
    cout << endl ;
  }
}
```

Pointer to a Pointer

Array of Pointers

Array of Pointers

```
char *myArray [ 10 ] ;
```

myArray is an array of 10 pointer to character

Initialization

```
char *myArray [ ] = { “Amir ” , “ Jahangir ” } ;
```

Storing Pointers in Array of Pointers

```
int  *p1 , *p2 , *p3 ;
int *b [ 3 ] ;
b [ 0 ] = p1 ;
b [ 1 ] = p2 ;
b [ 2 ] = p3 ;
```

Command Line Arguments

argc

argv

‘argc’ stands for a count of the number of arguments

‘argv’ stands for a vector of arguments

Example 4

```
main ( int argc , char **argv )
{
 cout << argc << "\n“ << *argv ;
}
```

Example 5

```
main ( )
{
 const char *suit [ 4 ]= { "Spades" , "Hearts" ,      "Diamonds" ,
    "Clubs" } ;
 const char *face [ 13 ] = { "Ace" , "Deuce" , "Three" , "Four",
                    "Five" , "Six" , "Seven" , "Eight" ,
          "Nine" , "Ten" , "Jack" , "Queen" , "King" } ;

 int deck [ 4 ] [ 13 ] = { 0 } ;

 srand ( time ( 0 ) ) ;
 shuffle ( deck ) ;
 deal ( deck , face , suit ) ;

}
```

Shuffle Functions

```
void shuffle ( int wDeck [ ] [ 13 ] )
{
 int row , column , card ;

 for ( card = 1 ; card <= 52 ; card ++ )
 {
  do
  {
   row = rand ( ) % 4 ;
        column = rand ( ) % 13 ;
         }
  while( wDeck[ row ]| column ] != 0 ) ;
  wDeck [ row ] [ column ] = card ;
 }
}
```

Deal Function

```
void deal ( const int wDeck [ ] [ 13 ] , const char *wFace [ ] , const char *wSuit [ ] )
{
 int card , row , column ;
 for ( card = 1 ; card <= 52 ; card ++ )
 {
  for ( row = 0 ; row <= 3 ; row++ )
  {
     for ( column = 0 ; column <= 12 ; column ++ )
      {
   if ( wDeck [ row ] [ column ] == card )
   // Print the face and suit  of the card
   // break out of loops
       }
         }
 }
}
```

What we have done today

- Multi-dimensional Arrays
- Pointers to Pointers
- Arrays of Pointers
- Command Line Arguments
- Comprehensive Example

VU
Virtual University